National University of Singapore
EE4002R CA2 Report:

Deep Learning for LDPC Code Decoding in Digital Communications

Academic Year 2024/2025 Semester 1

Student Name	**Matriculation Number**
Leong Kah Sheng	A0233541N

1. Abstract

This report covers an overview of regular Low Density Parity Check (LDPC) codes and the traditional method of decoding them. A machine learning learning approach, specifically deep learning model such as a Convolutional Neural Network (CNN) to explore the possibility of achieving a better performance to the traditional methods of decoding LDPC codes via belief propagation. The model is tested on short and long bits and compared against the performance of the LDPC graph using.

2. Introduction

Error correction is essential in digital communication to ensure that data transmitted across noisy channels remains accurate and reliable. Without error correction, noise and interference in the communication channel can lead to data corruption, which degrades the overall system performance. Low-Density Parity-Check (LDPC) codes are one of the most effective error-correcting codes widely used in high-performance applications, including satellite communications, 5G networks, and data storage. LDPC codes are structured using a sparse parity-check matrix that allows efficient error correction through algorithms like belief propagation. Accurate decoding of these codes is critical because even small improvements in Bit Error Rate (BER) can lead to significant enhancements in communication reliability and quality.

Deep learning, particularly Convolutional Neural Networks (CNNs), provides a promising alternative for LDPC decoding. CNNs can automatically learn complex patterns from data, making them highly adaptable to different noise levels and capable of generalising across diverse channel conditions. Additionally, CNNs can offer faster inference times after training, making them suitable for applications where real-time decoding is needed. By applying a CNN model, we can potentially improve the robustness and efficiency of LDPC decoding.

The goal of this report is to apply deep learning, specifically CNNs, to the task of LDPC decoding and to assess the performance of this approach in comparison to traditional decoding methods like belief propagation. By exploring CNN-based decoding, we aim to determine if deep learning can provide more efficient and accurate decoding solutions for LDPC codes in high-noise environments.

Main Contributions:

1. Benchmarking Traditional Decoding: We implemented and tested the message-passing algorithm (belief propagation) using a parity-check matrix from David McKay's website. This involved BPSK modulation of the codewords, simulating Gaussian white noise, and plotting BER vs. SNR performance to establish a baseline.

2. Development of a Deep Learning-based Decoder: A Convolutional Neural Network (CNN) was trained to explore the possibility of better decoding performance. The CNN model was trained to reduce BER by learning complex noise patterns, with experiments conducted on both short and long codewords to evaluate its capability.

3. Performance Comparison: Our CNN-based model was compared to the traditional method of decoding via belief propagation and the performance of uncoded codewords.

Overview of LDPC Codes

LDPC codes are a type of linear block code designed for error correction in digital communication, particularly in noisy environments. They are defined by a matrix, H. Specifically, this matrix H is known as a sparse parity-check matrix. “Sparse” refers to the matrix having more zeros than ones in each row and column, making them "low-density" or "sparse." This matrix is the key to encoding and decoding processes, as it determines the relationship between the bits that are being sent. The parity refers to an error-detection method to ensure that data is transmitted accurately. It involves adding an extra bit, called the parity (or check) bit, to a group of data bits. There are two main types of parity:

- Even Parity: The parity bit is set so that the total number of 1s in the data (including the parity bit) is even. For example, if the data is `1011` (which has three 1s, an odd number), an extra `1` is added to make the count of 1s even: `10111`.
- Odd Parity: The parity bit is set so that the total number of 1s is odd. For the data `1011`, a `0` would be added to keep the number of 1s odd: `10110`.

When bits are received, the number of “1”s (including the parity bit) are counted. If the count does not match the **expected** parity (even or odd), it means an error has likely occurred during transmission.

Encoding with LDPC Codes

The encoding process begins with a message vector, u (a 1 x k vector), which is transformed into v (a 1 x n vector) using a generator matrix, G of size k x n. The codeword v is obtained by the matrix multiplication of u and G:

$$v = uG$$

This results in a codeword that includes both data bits and redundant bits, allowing for error correction later on.

Decoding and Error Detection

During the transmission of data, errors may occur due to noise in the channel. To detect errors, LDPC codes calculate a syndrome S using the parity-check matrix H :

$$S = vH^T, \text{ where } H^T \text{ is the transpose of } H$$

If S = 0, it means the codeword is highly likely to be error-free. If S is not 0, an error has been detected. The syndrome helps identify potential error patterns that can then be corrected.

Structure of LDPC Codes

LDPC codes can be regular or irregular. For this study, the LDPC code used is regular, which means these codes have a fixed row and column weight. Weight refers to the number of ones in each row and column

of H, which are often constructed by dividing the parity-check matrix into parts and shuffling columns randomly.

Tanner Graph Representation

A Tanner graph is a representation of the parity-check matrix H. It is a bipartite graph with two types of nodes: variable and check nodes. Variable nodes represent the code bits (corresponding to columns in H). While check nodes represent the parity-check constraints (corresponding to rows in H).

Each "1" in the parity-check matrix defines an edge between a variable node and a check node, showing the relationships that help detect and correct errors. The girth of the graph (the shortest cycle) is a critical factor in the performance of LDPC codes, as longer cycles generally improve error-correction ability.

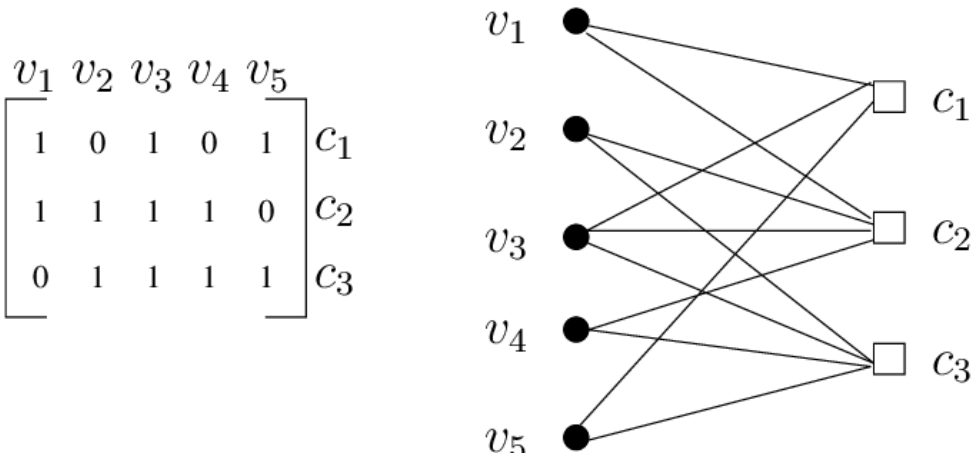


Figure A: Tanner Graph representation of a parity check matrix

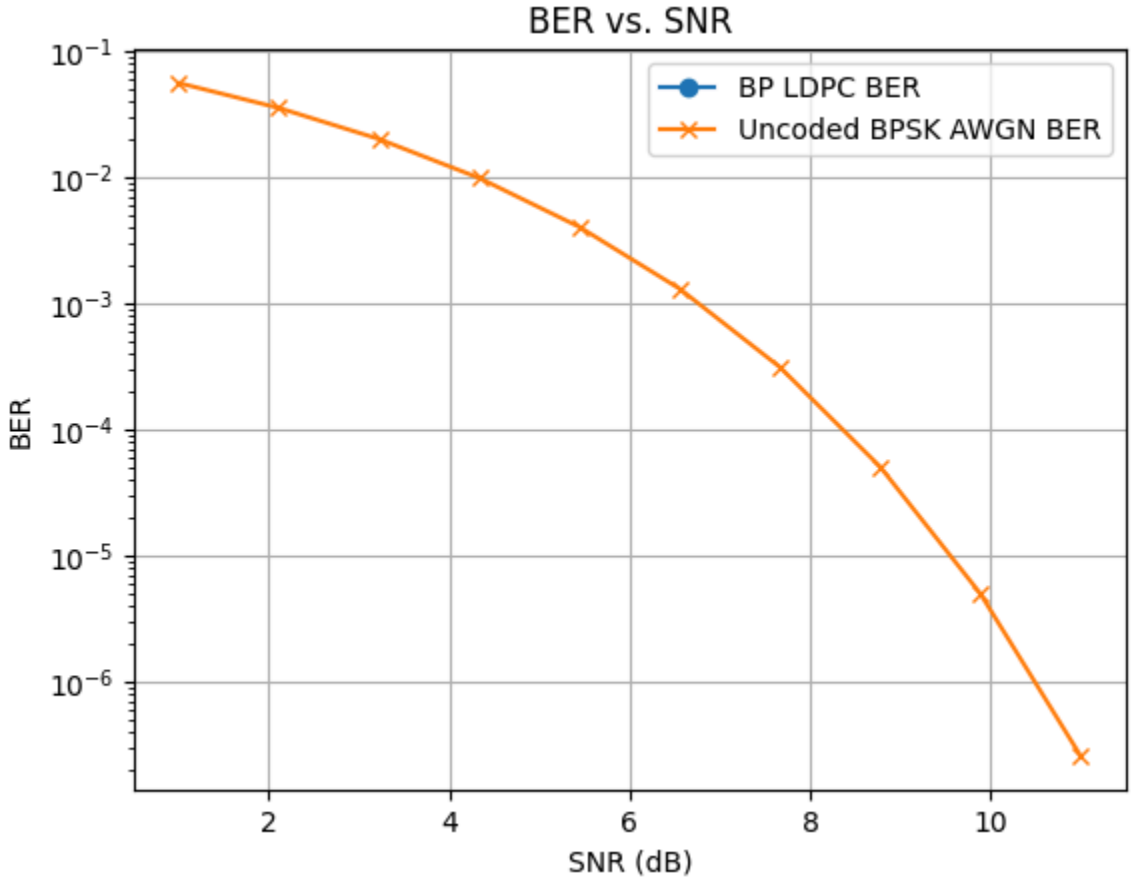
Belief Propagation for Decoding

Traditional LDPC decoding is performed via the belief propagation algorithm. This iterative algorithm passes messages between variable nodes and check nodes. First, check nodes send messages (which are the beliefs the check node has about the connected variable nodes. In this case, the beliefs are whether each bit is more likely to be “1” or “0”) to the connected variable nodes. Variable nodes then adjust their value based on the messages received from connected check nodes. This process continues until either the messages converge (all connected nodes agree on the value) or the maximum number of iterations is reached. Belief Propagation (BP) helps correct errors by refining the likelihood of each bit’s value in multiple iterations, ultimately allowing the decoder to reconstruct the original message accurately.

2.1 Implementation of BP algorithm

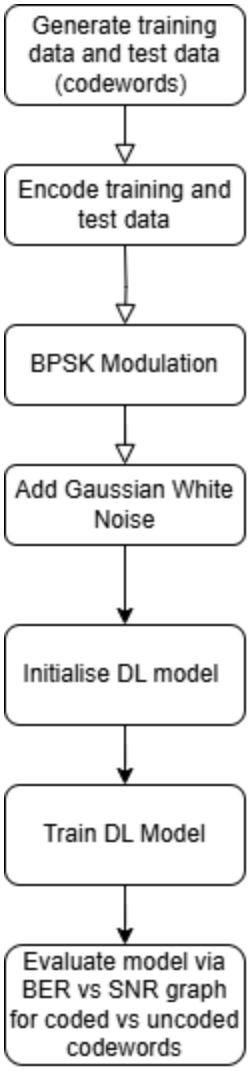
We implemented the BP algorithm and used a known parity check matrix from David MacKay’s website H. A codeword with all “0”s was then passed as the input into the decoder to ensure that the message passing algorithm was working correctly. Within one to two iterations, the 48-bit long BPSK modulated codeword with AGWN noise at various SNR (1 dB to 10 dB) was decoded with a BER of zero. The

details of the implementation can be found in [5].



3. Methodology

The model was trained using a data driven method, which involves generating codewords for the model to learn instead of using the parity check matrix to construct the architecture of the neural network. This means that we generate all possible codewords for a codeword of k-bits. Then, we encode them using a known parity check matrix (obtained from David Mackay's website). BPSK modulation followed by AWGN noise is then added to simulate a noisy channel. The CNN model is then training by passing the encoded data as inputs in batches, which are determined by the batch size. The model is then evaluated by generating another set of codewords. The following flow chart displays the implementation process:



More details of the implementation can be found in Appendix A.

3.1 Model Architecture:

3.1.1 Convolutional Layers

Layer 1: 1 input channel, 32 output channels, kernel size of 2, and padding of 1. This layer is followed by batch normalization and a ReLU activation.

Layer 2: 32 input channels, 64 output channels, kernel size of 2, and padding of 1, followed by batch normalization and ReLU activation.

Layer 3: 64 input channels, 128 output channels, kernel size of 2, and padding of 1, followed by batch normalization and ReLU activation.

3.1.2. Fully Connected (FC) Layers

FC Layer 1: Linear layer with an input size of `128 * encoded_length` and an output size of 256, followed by ReLU activation and dropout.

FC Layer 2: Linear layer with an input size of 256 and output size matching `decoded_length`, followed by a sigmoid activation to produce the final output.

3.1.3 Activation and Regularisation

Batch Normalisation: Applied after each convolutional layer to stabilise and accelerate training by normalising feature distributions.

Dropout: A dropout layer with a rate of 0.3 is added after the first fully connected layer to prevent overfitting.

Activation Functions: ReLU is used after each batch-normalised convolutional layer and the first fully connected layer. A sigmoid function is applied in the final layer for binary decoding.

3.2 Training Process

Following is the algorithm for the training process of the model:

For each **epoch**:

- For each **batch**:
    - Perform a **forward pass** to generate predictions.
    - **Compute loss** based on predictions vs. targets.
    - **Backpropagate** to calculate gradients.
    - **Update parameters** using the optimizer.
    - Track batch loss.
- Log **epoch loss** to monitor progress.

Repeat until all epochs are complete, then evaluate or save the model.

4. Results and Analysis

4. 1 Performance Metrics

In digital communications, two key metrics are often considered: **Bit Error Rate (BER)** and **Signal-to-Noise Ratio (SNR)**.

Bit Error Rate is a metric that quantifies the accuracy of the model by measuring the percentage of bits incorrectly predicted by the model compared to the total number of bits transmitted.

A lower BER indicates better performance, as it means fewer bits are incorrectly decoded. A BER of 0.01, for example, means that 1% of the bits were received in error. BER directly measures the decoding

effectiveness of the model in terms of actual bit errors. It's particularly useful because even a small BER can significantly impact communication quality in practical systems.

Signal-to-Noise Ratio is a measure of the strength of the desired signal relative to background noise. In the context of digital communication, SNR helps determine how noisy the channel is, which can affect the accuracy of decoding.

A higher SNR indicates a clearer signal with less noise, thus making it easier to decode. An SNR of 20 dB or higher generally means that the signal is much stronger than the noise, leading to potentially better decoding performance. SNR is often used to simulate real-world conditions, allowing researchers to test how well a model can perform in different noise environments. Models are usually tested at various SNR levels to assess their robustness under different noise conditions.

4.2 Performance Graphs:

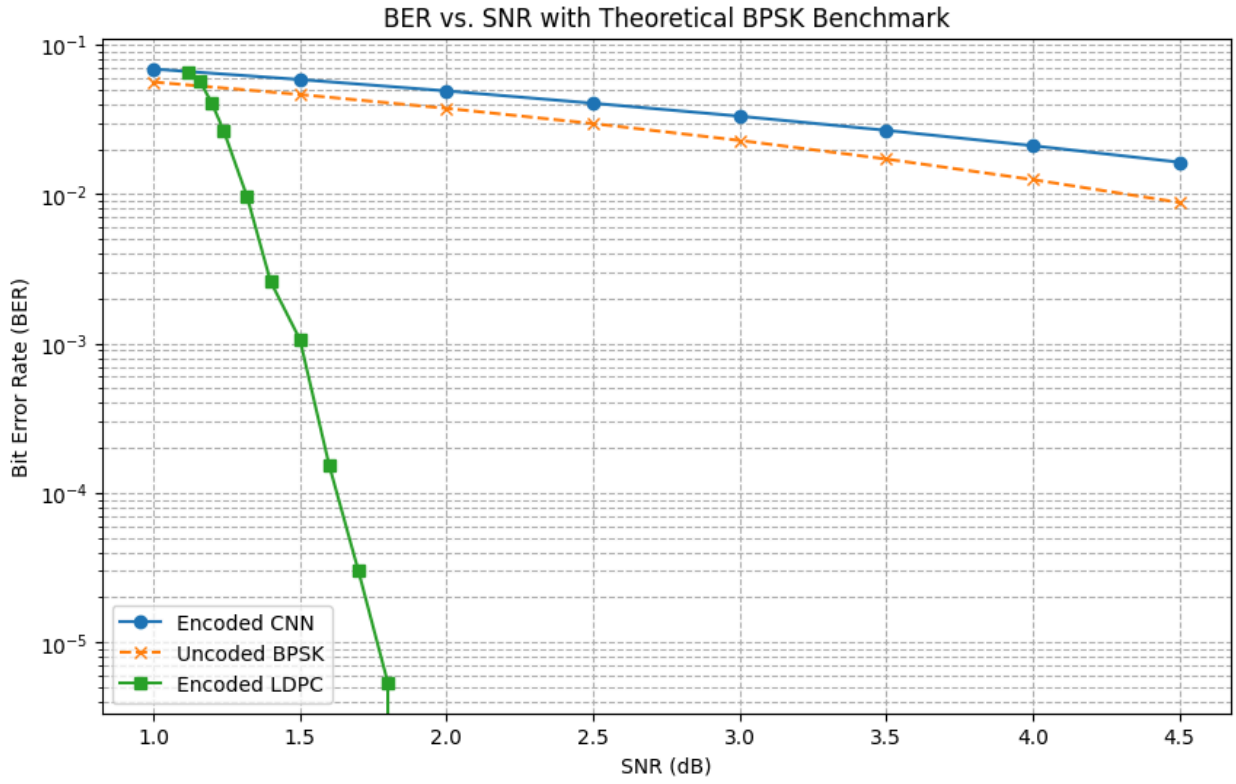


Figure A: Comparison of traditional method ("Encoded LDPC"), trained model ("Encoded CNN") and uncoded model ("Uncoded LDPC")

4.3 Analysis of Results:

The performance of the model is outperformed by the uncoded codewords and the encoded LDPC. Theoretically, we expect the CNN model to perform better than the uncoded graph. This is because encoded codewords using parity check matrix were passed in as input when training the model.

A possible reason could be due to covering only a subset of all possible k-bit long codewords. The full set of codewords consist of 2^k combinations. In our model, k = 48, which is a very large number. Our model took in a total of 6.4 million potentially different codewords. While it seems a lot, it is relatively small compared to the total number of possible keywords when k = 48. This could be attempted if a data driven model such as the CNN model continues to be used in the future.

Another possible reason might be that the model is not suitable for decoding LDPC codes. In LDPC codes, the relationships of bits is determined by their connections, which is determined by the parity check matrix. The convolutional model uses a kernel, which groups the data that are near to each other. Through this it attempts to capture local relationships between the data by averaging the data within the kernel. This assumes that data within the kernel are related. However, we know that is not necessarily true. The first bit and last bit of a codeword could be connected to the same check node. Thus, this could contribute to the poor training of the model.

5. Discussion

5.1 Advantages of CNN-Based LDPC Decoding: Summarise the key benefits, including adaptability to different noise conditions and potentially faster inference.

5.2 Limitations:

The primary challenge was trying to improve the performance of the model. There was an exponential increase in the number of combinations for every larger k-bit codeword that was used. This led to a huge computational complexity in generating all combinations of a k-bit codewords. To combat this problem, we leveraged the numpy function to generate pseudo random, k-bit long codewords. We also needed to ensure that the seed used for the generation of training data was randomised. Although this method might not allow us to generate all combination of data for a 48-bit codeword, we can argue that it is highly unlikely that two set of generated codewords are repeated. Thus this means during testing, the model is exposed to codewords it has not seen before.

5.3 Comparative Analysis:

The CNN-based decoding falls short compared to traditional techniques such as Belief Propagation.

Tried using another optimiser (SGD), changing the batch size, changing learning rate, changing epoch, loss function. However, the model's test accuracy did not improve.

6. Future Directions

Model Improvements: Future improvements might include training the network on all possible codewords. Instead of a data-driven model, an architectural approach which involves constructing the neural network using the parity check matrix may result in improvements in BER vs SNR performance of the model. In addition, different neural network models could be attempted if we are unable to obtain satisfactory performance after using the traditional methods of decoding as inspiration.

7. Conclusion

This study investigated the use of a Convolutional Neural Network (CNN) for decoding Low-Density Parity-Check (LDPC) codes, comparing its performance to traditional decoding methods such as belief propagation. Although the CNN demonstrated adaptability to various noise conditions and offered the potential for faster inference, its performance in terms of Bit Error Rate (BER) fell short of established LDPC decoding algorithms. This limitation is likely due to the CNN's focus on local data relationships, which may not effectively capture the complex, non-local dependencies defined by the LDPC parity-check matrix.

This study showcased both the advantages and obstacles of applying deep learning to error correction in digital communication. Future research will focus on improving the CNN model by integrating structural aspects of LDPC codes, like the parity-check matrix, to more effectively capture bit dependencies. By merging neural networks with traditional techniques or developing architectures influenced by message-passing algorithms, the goal is to achieve more accurate decoding, potentially making CNN-based decoding a robust solution for high-noise conditions.

8. References

[1] C. Lucibello, F. Pittorino, G. Perugini, and R. Zecchina, "Deep learning via message passing algorithms based on belief propagation," *Machine Learning Science and Technology*, vol. 3, no. 3, p. 035005, Jun. 2022, doi: 10.1088/2632-2153/ac7d3b.

[2] N. Devroye *et al.*, "Interpreting Deep-Learned Error-Correcting codes," *2022 IEEE International Symposium on Information Theory (ISIT)*, Jun. 2022, doi: 10.1109/isit50566.2022.9834599.

[3] A. Haroon, F. Hussain, & M.R. Bajwa, "Decoding of Error Correcting codes Using Neural Networks", 2013

[4] I. Be'ery, N. Raviv, T. Raviv, and Y. Be'ery, "Active deep decoding of linear codes," *arXiv (Cornell University)*, Jan. 2019, doi: 10.48550/arxiv.1906.02778.

[5] K. S. Leong, EE4002R-Deep-Learning-for-Communications. GitHub Repository. Available: https://github.com/BananaFalls/EE4002R-Deep-Learning-for-Communications

9. Appendix

Appendix A:

1. Initialise parameters: decoded_length, encoded_length, learning_rate, epochs, batch_size, etc.

2. Define function `generate_data(batch_size, snr_db)`

- Generate random binary data of size (batch_size, decoded_length)

- Encode data using generator matrix G

- Modulate encoded data with BPSK

- Add AWGN to modulated signal based on SNR

- Return noisy signal and original data

3. Define class `ImprovedCNNDecoder`

- Initialise convolutional, batch normalisation, fully connected layers, and dropout

- Define forward pass:

a. Unsqueeze input to add channel dimension

b. Apply conv layers with batch normalization and ReLU activation

c. Flatten output

d. Apply fully connected layers with ReLU, dropout, and Sigmoid activation

e. Return output

4. Training algorithm

For each epoch:

a. Initialise epoch_loss to 0

b. For each batch in num_batches:

i. Randomly select SNR (snr_db) for augmentation

ii. Generate inputs and targets using `generate_data`

iii. Zero gradients

iv. Forward pass: get model outputs

v. Compute loss with BCELoss

vi. Backpropagate and update weights

vii. Accumulate loss for the batch to epoch_loss

c. Print average epoch loss

5. Instantiate model, criterion, and optimizer

6. Call `train_model(model, optimizer, criterion)`